502094 - Software Deployment, Operations And Maintenance

Module 13: Monitoring, Logging & Observability

Lecturer: MSc. Mai Van Manh

Email: maivanmanh@tdtu.edu.vn

Module Overview

- Modern systems **will fail** — observability helps us understand **why**
- Focus of this module:
  - Concepts: **monitoring, logging, observability, tracing**
  - Tools: **Prometheus, Grafana, Loki, Promtail**
  - Basic introduction to **OpenTelemetry** and alerting
- Style:
  - Theory → concrete examples (Node.js / Spring Boot / Nginx / EC2)
- Goal:
  - Students can design an **observability stack** for a simple web system

1. Introduction to Observability

- **Observability**:
  - Ability to **understand internal state** of a system **from the outside**
- Not only “is it up?” but:
  - Why is it slow?
  - Which service is failing?
  - Which users are affected?
- In DevOps:
  - Observability is a **first-class capability**, not an afterthought

Observability vs Monitoring vs Logging

- **Monitoring**:
  - Continuous measurement of system metrics (CPU, latency, errors)
- **Logging**:
  - Detailed record of events (requests, errors, business events)
- **Observability**:
  - Combines metrics, logs, traces to answer **new questions**
- Analogy:
  - Monitoring: vital signs
  - Logging: detailed medical notes
  - Observability: full diagnostic toolkit

The Three Pillars of Observability

- **Metrics**:
  - Numeric time series (e.g., `http_requests_total`, CPU usage)
- **Logs**:
  - Timestamped events, often text or JSON
- **Traces**:
  - End-to-end record of a **single request** across multiple services
- Together:
  - Metrics tell you **that** something is wrong
  - Logs tell you **what** happened
  - Traces show you **where** it happened

Why Observability Matters

- Modern systems:
  - Multiple services (microservices)
  - Containers and orchestrators (Docker, Kubernetes)
  - Cloud infrastructure (AWS, GCP, Azure)
- Failures are:
  - Partial, intermittent, cross-service
- Observability enables:
  - Faster **incident detection**
  - Faster **root-cause analysis**
  - Better **postmortems** and learning

Challenges in Modern Architectures

- **Microservices**:
  - Many small services, each with its own logs and metrics
- **Containers**:
  - Short-lived, ephemeral pods/containers
  - Logs disappear when container is deleted
- **Multi-node systems**:
  - Requests jump between nodes and regions
- Without proper observability:
  - “It works on my machine” becomes “It sometimes fails in production”

Observability, Debugging & Alerting

- **Debugging**:
  - Process of finding and fixing specific issues
- **Alerting**:
  - Notifies humans (Slack, email, PagerDuty) when conditions are met
- **Observability**:
  - Provides data and tools to support debugging and alerting
- Pipeline:
  i. Monitoring detects anomaly
  ii. Alerts notify on-call engineer
  iii. Observability tools support investigation and fix

2. Monitoring Concepts & Architecture

- Focus:
  - **Black-box vs white-box monitoring**
  - **Push vs pull** models
  - TSDB, SLO/SLI/SLA
  - Golden signals and health checks
- Tools:
  - Prometheus, Node Exporter, application metrics

Black-Box vs White-Box Monitoring

- **Black-box**:
  - Observes system **from the outside**
  - Example:
    - Ping an HTTP endpoint every 30 seconds
  - Answers:
    - Is the service reachable? What is response time?
- **White-box**:
  - Uses **internal metrics** from the application
  - Example:
    - `http_requests_total`, DB query latency
  - Answers:
    - What is happening inside the app?

Push vs Pull Monitoring Models

- **Push**:
  - Applications **send metrics** to a collector
  - Example: some cloud monitoring agents
- **Pull** (Prometheus model):
  - Prometheus **scrapes metrics endpoints**
  - Applications expose `/metrics` HTTP endpoint
- Trade-offs:
  - Push: good for mobile/IoT, behind NAT
  - Pull: simple for server-side, easier to control scrape rate

Time-Series Databases (TSDB)

- TSDB stores metrics as:
  - `(timestamp, value, labels)`
- Examples:
  - Prometheus TSDB, Cortex, Thanos, VictoriaMetrics
- Why TSDB?
  - Efficient storage for **append-only** time series
  - Fast aggregation and queries (PromQL)
- Use cases:
  - Plot CPU usage over 1 week
  - Alert if error rate exceeds threshold

SLOs, SLIs, SLAs (Simple Intro)

- **SLI** (Service Level Indicator):
  - Measurable value: e.g. “99.9% of requests < 300 ms”
- **SLO** (Service Level Objective):
  - Target for SLI over time
- **SLA** (Service Level Agreement):
  - Contract with customers (often includes penalties)
- Beginners’ view:
  - Start with SLI/SLO as **internal targets** before SLAs

The “Four Golden Signals”

- For any user-facing service, monitor:
  - **Latency**: how long each request takes
  - **Traffic**: number of requests (RPS/QPS)
  - **Errors**: error rate (5xx, timeouts)
  - **Saturation**: how “full” the system is (CPU, memory, queue length)
- Example:
  - For a Node.js API:
    - Add metrics for latency, request count, HTTP status, event loop lag

Health Checks & Probes

- Simple HTTP health endpoint:

```
curl -i http://api.example.internal/health
```

- Kubernetes:
  - **Liveness probe**: should the container be restarted?
  - **Readiness probe**: is the pod ready to receive traffic?
- Example deployment snippet:

```
livenessProbe:
  httpGet:
    path: /health
    port: 3000
readinessProbe:
  httpGet:
    path: /ready
    port: 3000
```

3. Metrics with Prometheus

- Prometheus is:
  - **Open-source metrics** & alerting toolkit
  - Designed for cloud-native systems
- Key strengths:
  - Pull model (scrape)
  - Powerful query language **PromQL**
  - Integrates with Grafana, Alertmanager
- In this module:
  - Use Prometheus to monitor Node.js, Spring Boot, Nginx, EC2

3.1 Core Prometheus Concepts

- **Target**:
  - Endpoint that Prometheus scrapes (e.g. `http://app:9100/metrics`)
- **Job**:
  - Group of targets with same purpose
- **Metrics format**:
  - Text-based, human-readable

```
http_requests_total{method="GET",code="200"} 1027
```

- Common exporters:
  - **Node Exporter** for OS metrics
  - **cAdvisor** for containers

PromQL Basics

- PromQL operators:
  - Aggregations: `sum()`, `avg()`, `max()`, `min()`
  - Rate functions: `rate()`, `irate()`
- Example queries:

```
# RPS for all HTTP 200
rate(http_requests_total{code="200"}[5m])

# CPU usage per instance
avg by (instance) (rate(node_cpu_seconds_total[5m]))
```

- Filters:
  - `{label="value"}` to select series

Prometheus Configuration Example

- `prometheus.yml` excerpt:

```yaml
global:
  scrape_interval: 10s

scrape_configs:
  - job_name: "node_exporter"
    static_configs:
      - targets:
          - "node1:9100"
          - "node2:9100"

  - job_name: "booking-api"
    static_configs:
      - targets:
          - "booking-api:9101"
```

- Prometheus will:
  - Scrape metrics every 10s

3.2 Installing and Running Prometheus (Docker)

- Simple `docker-compose.yml`:

```yaml
version: "3"
services:
  prometheus:
    image: prom/prometheus:latest
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml
    ports:
      - "9090:9090"
```

- Commands:

```
docker compose up -d
```

- Access UI:
  - `http://localhost:9090`

Adding Targets to Prometheus

- Add a Node Exporter target:

```yaml
scrape_configs:
  - job_name: "node_exporter"
    static_configs:
      - targets:
          - "host.docker.internal:9100"
```

- Node Exporter:

```bash
docker run -d --name node_exporter     -p 9100:9100     prom/node-exporter
```

- Check target status:
  - Prometheus UI → Status → Targets

3.3 Exporters Overview

- **Node Exporter**:
  - Export CPU, memory, disk, network metrics
- **Blackbox Exporter**:
  - Probe HTTP, TCP, DNS, ICMP:
    - “Can we reach this endpoint?”
- **Application-specific exporters**:
  - Postgres, MySQL, Redis, Nginx, etc.
- Key idea:
  - Exporters expose metrics; Prometheus collects

Application Metrics: Node.js

- Using `prom-client`:

```
npm install prom-client express
```

- Basic code:

```
const express = require("express");
const client = require("prom-client");

const app = express();
const register = new client.Registry();

const httpRequests = new client.Counter({
  name: "http_requests_total",
  help: "Total HTTP requests",
  labelNames: ["method", "route", "status"]
});

register.registerMetric(httpRequests);
client.collectDefaultMetrics({ register });

app.use((req, res, next) => {
  res.on("finish", () => {
    httpRequests.inc({ method: req.method, route: req.path, status: res.statusCode });
```

Application Metrics: Spring Boot

- Dependencies:
  - `spring-boot-starter-actuator`
  - `micrometer-registry-prometheus`
- `application.yml`:

```yaml
management:
  endpoints:
    web:
      exposure:
        include: health,info,prometheus
  endpoint:
    prometheus:
      enabled: true
```

- Prometheus target:

```yaml
- job_name: "spring-boot"
```

4. Visualization with Grafana

- Grafana:
  - Visualization platform for time-series data
  - Supports many data sources:
    - Prometheus, Loki, Elastic, CloudWatch, etc.
- In our stack:
  - Prometheus = metrics store
  - Grafana = dashboards + alerts

4.1 Grafana Basics

- Core concepts:
  - **Data source**: where data comes from (e.g. Prometheus)
  - **Dashboard**: collection of panels, often per system or team
  - **Panel**: a single chart/table/stat
  - **Variables**: dynamic filters (e.g. instance, environment)
- Alerts:
  - Modern Grafana uses **Unified Alerting**:
    - Create alert rules attached to queries

Running Grafana with Docker

- `docker-compose.yml` snippet:

```yaml
services:
  grafana:
    image: grafana/grafana:latest
    ports:
      - "3000:3000"
    environment:
      - GF_SECURITY_ADMIN_USER=admin
      - GF_SECURITY_ADMIN_PASSWORD=devops123
```

- After start:
  - Login at `http://localhost:3000`
  - Add Prometheus as data source:
    - URL: `http://prometheus:9090`

Building a Basic Dashboard

- Steps:
  i. Create a dashboard
  ii. Add a panel: “CPU usage per instance”
  iii. Query:

```
100 - (avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)
```

  iv. Visualization: Time series
  v. Add thresholds (e.g. > 80% = red)
- Repeat for:
  - Memory usage
  - Disk I/O
  - Network traffic

Visualizing Application Metrics

- Node.js example:
  - Query requests per second:

```
sum by (route) (rate(http_requests_total[1m]))
```

- Spring Boot:
  - Error rate:

```
rate(http_server_requests_seconds_count{outcome="SERVER_ERROR"}[5m])
```

- Dashboard design:
  - Group panels by:
    - System metrics
    - Application metrics
    - Database metrics

Dashboard Best Practices

- Keep dashboards:
  - **Clear**: avoid too many panels on one screen
  - **Consistent**: same color coding across dashboards
  - **Actionable**: each chart should help answer a question
- Typical dashboards:
  - API overview (latency, RPS, errors)
  - Infrastructure overview (CPU, memory, disk)
  - Business KPIs (orders, signups)

5. Logging Systems

- Focus:
  - Logging concepts and log levels
  - Centralized logging stacks:
    - Loki, ELK
  - Log collection with Promtail
- Goal:
  - Avoid logging only to “tail -f” on production servers

5.1 Logging Concepts

- **Unstructured logs**:
  - Plain text, difficult to search reliably
- **Structured logs**:
  - Key-value or JSON, easy to query
- Log levels:
  - `DEBUG`, `INFO`, `WARN`, `ERROR`, `FATAL`
- Example JSON log:

```
{
  "level": "error",
  "msg": "Failed to save order",
  "orderId": 123,
  "userId": 42,
  "timestamp": "2025-10-10T10:03:00Z"
}
```

Why Centralized Logging?

- Problems with local logs:
  - Lost when server or container is recreated
  - Need SSH to each server to inspect logs
- Centralized logging:
  - All logs shipped to **one place**
  - Search across all services and instances
- Supports:
  - Incident investigations
  - Security incident analysis
  - Compliance requirements

Logging in Containerized Environments

- Containers:
  - Often write logs to **stdout/stderr**
  - Docker/Kubernetes capture logs
- Best practice:
  - Do not write logs only to files inside container
  - Use **log drivers** or **sidecars** (Promtail/Fluentd) to ship logs
- For Kubernetes:
  - Daemonset agents on each node

5.2 Centralized Logging Stack Options

- **Loki**:
  - Log aggregation system by Grafana Labs
  - Designed to work like “Prometheus for logs”
  - Efficient, label-based indexing
- **ELK Stack**:
  - Elasticsearch + Logstash + Kibana
  - Very powerful but heavier
- Cloud providers:
  - AWS CloudWatch Logs
  - GCP Cloud Logging
  - Azure Monitor

5.3 Loki + Promtail Overview

- Components:
  - **Loki**: log storage and query backend
  - **Promtail**: agent that ships logs to Loki
  - **Grafana**: UI to query logs (LogQL)
- For this course:
  - Use Docker Compose to run Loki + Promtail + Grafana
  - Collect logs from:
    - Node.js containers
    - Spring Boot containers
    - Nginx access logs

Loki + Grafana with Docker Compose

- Example `docker-compose.yml` snippet:

```yaml
services:
  loki:
    image: grafana/loki:2.9.0
    ports:
      - "3100:3100"
    command: -config.file=/etc/loki/local-config.yaml

  promtail:
    image: grafana/promtail:2.9.0
    volumes:
      - /var/log:/var/log
      - ./promtail-config.yml:/etc/promtail/config.yml
    command: -config.file=/etc/promtail/config.yml
```

- Grafana:
  - Add Loki as data source: `http://loki:3100`

Promtail Configuration Example

- `promtail-config.yml`:

```yaml
server:
  http_listen_port: 9080

clients:
  - url: http://loki:3100/loki/api/v1/push

scrape_configs:
  - job_name: "varlogs"
    static_configs:
      - targets:
          - localhost
        labels:
          job: "varlogs"
          __path__: /var/log/*.log
```

- Promtail:
  - Reads log files

Querying Logs with LogQL

- Basic query:

```
{job="varlogs"} |= "ERROR"
```

- Filter by label:

```
{app="booking-api", level="error"}
```

- Aggregate:

```
sum by (level) (rate({app="booking-api"}[5m]))
```

- Combine logs + metrics:
  - Use Grafana to link from metrics panels to relevant logs

6. Tracing (High-Level Introduction)

- **Distributed tracing**:
  - Observes a single request across multiple services
- Key concepts:
  - **Trace**: end-to-end journey of a request
  - **Span**: a single timed operation within the trace
  - **Context propagation**: passing trace IDs across services
- Helpful when:
  - Many microservices are involved
  - Latency is high and root cause unclear

OpenTelemetry Overview

- **OpenTelemetry (OTel)**:
  - Vendor-neutral standard for:
    - Metrics
    - Logs
    - Traces
  - Provides SDKs and auto-instrumentation
- Typical architecture:
  - Services → OTel SDK/agents → OTel Collector → backends (Jaeger, Tempo, etc.)
- In this course:
  - High-level concepts
  - Optional lab for traces

Example: Tracing a Web Request

- Flow:
  i. User calls `/checkout`
  ii. API calls:
    - Auth service
    - Payment service
    - Inventory service
  iii. Each service creates spans with a shared trace ID
- Tracing UI shows:
  - Timeline of spans
  - Which service slowed down the request
  - Errors at which step

7. Alerts & Notifications

- Alerts:
  - Automated checks that detect when something is wrong
- Goals:
  - Notify humans **only when necessary**
  - Avoid **alert fatigue**
- In this module:
  - Use Prometheus/Alertmanager or Grafana alerts
  - Send notifications to Slack/email in labs

7.1 Basic Alerting Concepts

- **Threshold alerts**:
  - e.g., CPU > 90% for 10 minutes
- **Rate-based alerts**:
  - e.g., error rate > 5% of requests
- Avoid:
  - Too many alerts
  - Alerts that cannot be acted on
- Good alerts:
  - Specific, actionable, tied to SLOs

7.2 Prometheus Alertmanager

- Prometheus:
  - Evaluates **alerting rules**
- Alertmanager:
  - Manages notifications:
    - Routing
    - Silencing
    - Grouping
- Example alert rule:

```yaml
- alert: HighErrorRate
  expr: rate(http_requests_total{code=~"5.."}[5m]) > 0.05
  for: 10m
  labels:
    severity: warning
  annotations:
```

Alertmanager Configuration Snippet

- `alertmanager.yml`:

```yaml
route:
  receiver: "slack-notifications"

receivers:
  - name: "slack-notifications"
    slack_configs:
      - api_url: https://hooks.slack.com/services/XXX/YYY/ZZZ
        channel: "#alerts"
        text: "{{ .CommonAnnotations.summary }}"
```

- Security:
  - Slack webhook URLs should be stored as **secrets**, not committed to Git

Grafana Unified Alerting

- Alternative to Alertmanager:
  - Create alert rules directly on Grafana panels
- Flow:
  i. Write Prometheus query
  ii. Add alert condition (e.g. `WHEN avg() OF query(A) > 0.8`)
  iii. Configure contact points (Slack, email)
- Good for:
  - Application-specific or dashboard-specific alerts

7.3 Notifying Teams

- Channels:
  - **Slack**: common in modern teams
  - **Email**: baseline, but easy to ignore
  - **PagerDuty / Opsgenie**:
    - For on-call rotation and escalation
- Principles:
  - Critical alerts: wake someone up
  - Non-critical: send to channel for review
  - Always include links to dashboards/logs in alert texts

8. Monitoring & Logging in Cloud Environments

- On AWS:
  - **CloudWatch** is the default monitoring/logging service
- Need to decide:
  - Use **CloudWatch only**
  - Or integrate CloudWatch + Prometheus/Grafana
- In this course:
  - Basic CloudWatch overview
  - Prometheus + node exporter approach on EC2

8.1 AWS CloudWatch Basics

- **CloudWatch Metrics**:
  - Built-in metrics for EC2, ELB, RDS, etc.
- **CloudWatch Logs**:
  - Centralized log storage (app logs, system logs)
- **CloudWatch Alarms**:
  - Alerts based on metric thresholds
- **Dashboards**:
  - Basic visualization within AWS console

When to Use CloudWatch vs Prometheus/Grafana

- **CloudWatch-only**:
  - Simple setups fully inside AWS
  - Teams that prefer managed services
- **Prometheus + Grafana**:
  - Complex, multi-cloud or hybrid environments
  - Need fine-grained metrics & custom alerting
- Mixed approach:
  - CloudWatch for base metrics, billing
  - Prometheus for app metrics and deep analysis

8.2 Integrating EC2 Metrics into Prometheus

- Install Node Exporter on EC2:

```
ssh ubuntu@EC2_PUBLIC_IP
sudo docker run -d --name node_exporter     -p 9100:9100     prom/node-exporter
```

- Prometheus config:

```
- job_name: "ec2-nodes"
  static_configs:
    - targets:
        - "EC2_PUBLIC_IP:9100"
```

- Security:
  - Use security groups to restrict access to port 9100
  - Ideally scrape via private IPs within VPC

9. Observability in Containers & Kubernetes (Preview)

- Kubernetes adds:
  - More layers to observe:
    - Pods, nodes, services, ingresses, controllers
- Typical stack:
  - Prometheus + cAdvisor + kube-state-metrics
  - Loki/ELK for logs
- Optional integration:
  - Service mesh (Istio, Linkerd) for detailed traffic metrics

cAdvisor & kube-state-metrics

- **cAdvisor**:
  - Collects container-level metrics:
    - CPU, memory, filesystem stats per container
- **kube-state-metrics**:
  - Exposes Kubernetes object states:
    - Deployments, pods, nodes, etc.
- Combined:
  - Provide rich picture of cluster health and workloads

Logging in Kubernetes

- Common approach:
  - Each node runs:
    - Promtail / Fluentd / Fluent Bit as DaemonSet
  - Agents collect:
    - Container stdout/stderr
    - Node system logs
  - Logs sent to:
    - Loki, Elasticsearch, or cloud logging
- Good practice:
  - Ensure logs include:
    - Pod name, namespace, container name, labels

Service Mesh Observability (Brief)

- Service mesh (e.g., Istio, Linkerd) provides:
  - Automatic metrics per service and route
  - Distributed tracing integrations
- Benefits:
  - No need to instrument every app manually for basic metrics
- Cost:
  - Added complexity and operational overhead
- At student level:
  - Understand concept, not necessarily operate a mesh yet

10. Hands-On Labs

- Lab 1 — Prometheus + Grafana
- Lab 2 — Application Metrics (Node.js & Spring Boot)
- Lab 3 — Centralized Logging with Loki + Promtail
- Lab 4 — Basic Alerting
- Lab 5 — Basic Tracing (optional bonus)
- Each lab:
  - Builds skills needed for **production-ready observability**

Lab 1 — Install Prometheus + Grafana

- Steps:
  i. Use Docker Compose to run Prometheus and Grafana
  ii. Configure Prometheus scrape configs for Node Exporter
  iii. Add Prometheus as data source in Grafana
  iv. Build a simple system metrics dashboard
- Commands:

```
docker compose up -d
```

- Expected:
  - Students can see CPU and memory graphs

Lab 2 — Monitor Node.js & Spring Boot

- Tasks:
  - Add `prom-client` to Node.js app
  - Enable `/actuator/prometheus` in Spring Boot
  - Configure Prometheus to scrape both apps
- Dashboard:
  - Requests per second per application
  - Error rate per endpoint
  - Latency histograms (if implemented)
- Goal:
  - Students understand **application-level metrics**

Lab 3 — Centralized Logging with Loki + Promtail

- Steps:
  i. Run Loki + Promtail + Grafana via Docker Compose
  ii. Configure Promtail to read:
    - Node.js logs
    - Spring Boot logs
    - Nginx access logs
  iii. Query logs in Grafana using LogQL
  iv. Filter logs by app, level, status code
- Outcome:
  - Single place to search across all logs

Lab 4 — Set Up Basic Alerting

- Tasks:
  - Create Prometheus or Grafana alert rules:
    - High CPU
    - High error rate
  - Configure alert notifications:
    - Slack webhook or email
  - Simulate overload:
    - Run `ab` or `wrk` to generate traffic
- Goal:
  - Experience full loop: metric → alert → notification

Lab 5 — Basic Tracing (Optional Bonus)

- Tasks:
  - Add OpenTelemetry auto-instrumentation to Node.js or Spring Boot
  - Run Jaeger or Zipkin via Docker
  - Send traces from app to tracing backend
  - Observe:
    - Per-endpoint traces
    - Latency breakdown per span
- Learning:
  - Traces as complement to logs & metrics

11. Best Practices for Observability

- Collect only meaningful data:
  - Avoid “log everything at debug” in production
- Use **structured JSON logs**
- Never store logs only on servers:
  - Ship them to centralized storage
- Build dashboards for:
  - System health
  - Application performance
  - Business metrics
- Keep alerts:
  - Actionable, deduplicated, prioritized

Observability as Code

- Treat observability artifacts like code:
  - Dashboards in JSON/Grafana provisioning
  - Alert rules in YAML
  - Prometheus configs in Git
- Benefits:
  - Version control
  - Code review
  - Reproducible environments
- Example:
  - Store `prometheus.yml`, `alerting-rules.yml`, `dashboards/` in the same repo as infrastructure code

Summary & Next Steps

- Today you learned:
  - Core observability concepts (metrics, logs, traces)
  - Prometheus + Grafana + Loki/Promtail stack
  - Basics of alerting and tracing
- Next:
  - Integrate observability into:
    - CI/CD pipelines
    - Infrastructure as Code
- Key message:
  - **You cannot operate modern systems safely without good observability.**